

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Algoritma Sorting”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 5 Juni 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Algoritma Sorting.....	3
2.2 Macam-Macam Algoritma Sorting yang dipelajari pada praktikum	3
2.3 Implementasi Stack pada Java.....	4
2.2.1 Class Merge Sort (MergeSort_2511533016)	4
2.2.2 Class Quick Sort (QuickSort_2511533016).....	7
2.2.3 Class Insertion sort (InsertionSort_2511533016).....	10
2.2.4 Class Bubble Sort GUI (BubbleSortGUI_2511533016).....	12
2.2.5 Class Merge Sort GUI (MergeSortGUI_2511533016)	15
BAB III PENUTUP	19
3.1 Kesimpulan.....	19
3.2 Saran.....	19
DAFTAR PUSTAKA.....	20
LAPORAN TUGAS PRAKTIKUM	21
1. Class Lagu (Lagu_2511533016)	21
2. Class Sorting (Sorting_2511533016)	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengolahan data, salah satu operasi yang paling sering dilakukan adalah mengurutkan sekumpulan data agar lebih mudah dikelola, dicari, maupun dianalisis. Kebutuhan ini muncul di hampir semua bidang, mulai dari sistem perbankan yang mengurutkan transaksi berdasarkan tanggal, aplikasi e-commerce yang menampilkan produk dari harga terendah ke tertinggi, hingga sistem akademik yang mengurutkan nilai mahasiswa. Tanpa adanya mekanisme pengurutan yang baik, proses pencarian dan pengelolaan data dalam jumlah besar akan menjadi sangat lambat dan tidak efisien.

Algoritma sorting atau algoritma pengurutan hadir sebagai solusi dari permasalahan tersebut. Terdapat berbagai macam algoritma sorting yang telah dikembangkan, masing-masing memiliki cara kerja, kelebihan, dan kekurangannya sendiri. Beberapa yang umum dipelajari antara lain Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, dan Quick Sort. Setiap algoritma memiliki karakteristik yang berbeda dalam hal kompleksitas waktu dan ruang, sehingga pemilihan algoritma yang tepat sangat bergantung pada kondisi dan kebutuhan data yang akan diurutkan. Oleh karena itu, pemahaman mendalam tentang algoritma sorting menjadi hal yang sangat penting bagi setiap mahasiswa yang mempelajari ilmu komputer dan struktur data.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 8 tentang Sorting adalah sebagai berikut:

1. Memahami konsep dasar dan cara kerja dari berbagai algoritma sorting yang umum digunakan dalam pemrograman.
2. Mampu mengimplementasikan algoritma sorting ke dalam kode program menggunakan bahasa pemrograman Java.
3. Melatih kemampuan berpikir logis dan analitis dalam memilih algoritma

4. sorting yang paling sesuai dengan kebutuhan dan karakteristik data yang ada.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 8 tentang Sorting adalah sebagai berikut:

1. Meningkatkan kemampuan dalam menulis program yang lebih efisien dan optimal dalam mengelola serta memproses data dalam jumlah besar.
2. Memberikan pemahaman yang lebih baik tentang bagaimana sebuah sistem komputer bekerja dalam mengurutkan dan mengorganisir data secara terstruktur.
3. Menjadi bekal penting dalam menghadapi permasalahan nyata di dunia kerja yang seringkali melibatkan pengolahan data dalam skala besar seperti di bidang teknologi, keuangan, maupun kesehatan.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Sorting atau pengurutan adalah proses menyusun sekumpulan data ke dalam urutan tertentu, baik dari nilai terkecil ke terbesar (ascending) maupun dari nilai terbesar ke terkecil (descending). Dalam ilmu komputer, sorting merupakan salah satu operasi fundamental yang sangat sering digunakan karena hampir semua proses pengolahan data membutuhkan data yang terurut agar lebih mudah diakses, dicari, dan dianalisis. Algoritma sorting sendiri adalah langkah-langkah sistematis yang digunakan oleh program untuk melakukan proses pengurutan tersebut secara otomatis dan efisien.

2.2 Macam-Macam Algoritma Sorting yang dipelajari pada praktikum

1. Bubble Sort

Bubble Sort adalah algoritma pengurutan paling sederhana yang bekerja dengan cara membandingkan dua elemen yang berdekatan secara berulang, kemudian menukarnya jika urutannya salah. Proses ini terus diulang hingga tidak ada lagi pertukaran yang perlu dilakukan. Dinamakan Bubble Sort karena elemen dengan nilai terbesar akan "menggelembung" ke posisi paling akhir seperti gelembung yang naik ke permukaan air. Algoritma ini mudah dipahami namun kurang efisien untuk data dalam jumlah besar karena kompleksitas waktunya $O(n^2)$.

2. Selection Sort

Selection Sort bekerja dengan cara mencari elemen terkecil dari seluruh data yang belum terurut, kemudian menukarnya dengan elemen di posisi paling depan. Proses ini diulangi terus menerus hingga semua elemen berada di posisi yang benar. Keunggulan Selection Sort adalah jumlah pertukaran yang dilakukan lebih sedikit dibanding Bubble Sort, namun kompleksitas waktunya tetap $O(n^2)$ sehingga masih kurang efisien untuk data berukuran besar.

3. Insertion Sort

Insertion Sort bekerja dengan cara mengambil satu elemen dari data yang belum terurut, kemudian menyisipkannya ke posisi yang tepat di antara elemen-elemen yang sudah terurut sebelumnya. Cara kerjanya mirip dengan bagaimana seseorang mengurutkan kartu di tangannya satu per satu. Algoritma ini sangat efisien untuk data yang sudah hampir terurut dan cocok digunakan pada data berukuran kecil, dengan kompleksitas waktu terbaik $O(n)$ dan terburuk $O(n^2)$.

2.3 Implementasi Stack pada Java

Pada praktikum pekan 8 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Bubble sort, Selection sort, Insertion sort di bawah ini kode program yang dituliskan pada praktikum pekan 8.

2.2.1 Class Merge Sort (MergeSort_2511533016)

```

1. package Pekan8_2511533016;
2.
3. public class MergeSort_2511533016 {
4.     void merge_2511533016(int arr_3016[],int l_3016, int
m_3016, int r_3016) {
5.         // find
6.         int n1_3016 = m_3016 - l_3016 + 1;
7.         int n2_3016 = r_3016 - m_3016;
8.
9.         int L_3016[] = new int [n1_3016];
10.        int R_3016[] = new int [n2_3016];
11.
12.        for (int i_3016 = 0; i_3016 < n1_3016; ++i_3016)
13.            L_3016[i_3016] = arr_3016[l_3016 + i_3016];
14.        for (int j_3016 = 0; j_3016 < n2_3016; ++j_3016)
15.            R_3016[j_3016] = arr_3016[m_3016 + 1 +
j_3016];
16.        int i_3016 = 0, j_3016 = 0;
17.
18.        int k_3016 = l_3016;
19.        while (i_3016 < n1_3016 && j_3016 < n2_3016) {
20.            if(L_3016[i_3016] <= R_3016[j_3016]) {
21.                arr_3016[k_3016] = L_3016[i_3016];
22.                i_3016++;
23.            } else {
24.                arr_3016[k_3016] = R_3016[j_3016];
25.                j_3016++;
26.            }
27.            k_3016++;

```

```

28.         }
29.
30.         while (i_3016 < n1_3016) {
31.             arr_3016[k_3016] = L_3016[i_3016];
32.             i_3016++;
33.             k_3016++;
34.         }
35.     }
36.
37.     void sort_2511533016(int arr_3016[], int l_3016, int
r_3016) {
38.         if(l_3016 < r_3016) {
39.             int m_3016 = (l_3016 + r_3016) / 2;
40.             sort_2511533016(arr_3016, l_3016, m_3016);
41.             sort_2511533016(arr_3016, m_3016 + 1,
r_3016);
42.             merge_2511533016(arr_3016, l_3016, m_3016,
r_3016);
43.         }
44.     }
45.
46.     static void printArray_2511533016(int arr_3016[]) {
47.         int n_3016 = arr_3016.length;
48.         for(int i_3016 = 0; i_3016 < n_3016; ++i_3016)
49.             System.out.print(arr_3016[i_3016] + " ");
50.         System.out.println();
51.     }
52.
53.     public static void main(String[] args) {
54.         int arr_3016[] = { 12, 11, 13, 5, 6, 7 };
55.         System.out.println("Sebelum terurut");
56.         printArray_2511533016(arr_3016);
57.         MergeSort_2511533016 ob_1002 = new
MergeSort_2511533016();
58.         ob_1002.sort_2511533016(arr_3016, 0, arr_3016.length
- 1);
59.         System.out.println("\nSesudah Terurut menggunakan
merge Sort");
60.         printArray_2511533016(arr_3016);
61.     }
62. }

```

```

Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
|

```

Gambar 2.2.1

Kode di atas adalah implementasi algoritma Merge Sort yang berada

dalam package Pekan7_2511533016. Kelas utamanya bernama MergeSort_2511533016 yang berisi dua fungsi utama untuk proses pengurutan yaitu mergeSort_3016 dan merge_3016, serta satu fungsi main untuk menjalankan program.

Fungsi mergeSort_3016 menerima tiga parameter yaitu array, indeks kiri, dan indeks kanan. Fungsi ini bekerja secara rekursif dengan cara pertama-tama mencari titik tengah array menggunakan rumus $(\text{left} + \text{right}) / 2$. Setelah titik tengah ditemukan, fungsi memanggil dirinya sendiri untuk bagian kiri yaitu dari indeks kiri sampai tengah, kemudian memanggil dirinya sendiri lagi untuk bagian kanan yaitu dari tengah tambah satu sampai indeks kanan. Proses pemecahan ini terus berlanjut hingga setiap bagian hanya memiliki satu elemen yang secara otomatis sudah dianggap terurut. Setelah kedua bagian selesai direkursi, fungsi merge_3016 dipanggil untuk menggabungkan kembali kedua bagian tersebut.

Fungsi merge_3016 bertugas menggabungkan dua bagian array yang masing-masing sudah terurut menjadi satu bagian yang terurut secara keseluruhan. Pertama program membuat dua array sementara yaitu L_3016 untuk menyimpan bagian kiri dan R_3016 untuk menyimpan bagian kanan, lalu menyalin data dari array asli ke kedua array sementara tersebut. Selanjutnya program membandingkan elemen-elemen dari kedua array sementara satu per satu, elemen yang lebih kecil dimasukkan terlebih dahulu ke array asli. Setelah perulangan perbandingan selesai, jika masih ada sisa elemen di salah satu array sementara yang belum dipindahkan maka sisa elemen tersebut langsung disalin ke array asli secara berurutan.

Pada fungsi main, program mendefinisikan array dengan enam elemen yaitu {12, 11, 13, 5, 6, 7} yang belum terurut. Sebelum proses pengurutan, program mencetak isi array awal dengan label "Sebelum terurut", kemudian memanggil fungsi mergeSort_3016 dengan parameter array, indeks awal 0, dan indeks akhir sesuai panjang array dikurangi satu. Setelah proses selesai, program mencetak kembali isi array yang kini sudah terurut dari nilai terkecil ke terbesar dengan label "Sesudah Terurut menggunakan merge Sort" sehingga menghasilkan output 5 6 7 11 12 13.

Kesimpulan : Kode program ini merupakan implementasi algoritma Merge Sort dalam bahasa Java, yang mendemonstrasikan cara mengurutkan sekumpulan data integer dari nilai terkecil ke terbesar (ascending) menggunakan pendekatan divide and conquer yaitu membagi array menjadi bagian-bagian kecil secara rekursif kemudian menggabungkannya kembali dalam urutan yang benar.

2.2.2 Class Quick Sort (QuickSort_2511533016)

```

1. package Pekan8_2511533016;
2.
3. public class QuickSort_2511533016 {
4.     static void swap_2511533016(int[] arr_3016, int i_3016, int
j_3016) {
5.         int temp_3016 = arr_3016[i_3016];
6.         arr_3016[i_3016] = arr_3016[j_3016];
7.         arr_3016[j_3016] = temp_3016;
8.     }
9.     // Metode tambahan ntuk mengatur pivot menggunakan Median-
Of-Three
10.    static void medianOfThree_2511533016(int[] arr_3016, int
low_3016, int high_3016) {
11.        int mid_3016 = low_3016 + (high_3016 - low_3016) /
2;
12.
13.        // urutan elemen low, mid, dan high
14.        if (arr_3016[low_3016] > arr_3016[mid_3016]) {
15.            swap_2511533016(arr_3016, low_3016,
mid_3016);
16.        }
17.        if (arr_3016[low_3016] > arr_3016[high_3016]) {
18.            swap_2511533016(arr_3016, low_3016,
high_3016);
19.        }
20.        if (arr_3016[mid_3016] > arr_3016[high_3016]) {
21.            swap_2511533016(arr_3016, mid_3016,
high_3016);
22.        }
23.        swap_2511533016(arr_3016, mid_3016, high_3016);
24.    }
25.    static int partition_2511533016(int[] arr_3016, int
low_3016, int high_3016) {
26.        //panggil fungsi medianofthree sebelum menentukan
pivot
27.        medianOfThree_2511533016(arr_3016, low_3016,
high_3016);
28.
29.        int pivot_3016 = arr_3016[high_3016]; // sekarang
arr[high] sudah berisi nilai median
30.        int i_3016 = (low_3016 - 1);
31.

```

```

32.         for (int j_3016 = low_3016; j_3016 <= high_3016 - 1;
33.             j_3016++) {
34.             //jika elemen saat ini lebih kecil dari atau
35.             sama dengan pivot
36.             if (arr_3016[j_3016] < pivot_3016) {
37.                 // increment indeks elemen yang lebih
38.                 kecil
39.                 i_3016++;
40.                 swap_2511533016(arr_3016, i_3016,
41.                                 j_3016);
42.             }
43.             swap_2511533016(arr_3016, i_3016 + 1, high_3016);
44.             return(i_3016 + 1);
45.         }
46.         static void quickSort_2511533016(int[] arr_3016, int
47.             low_3016, int high_3016) {
48.             if (low_3016 < high_3016) {
49.                 int pi_3016 = partition_2511533016(arr_3016,
50.             low_3016, high_3016);
51.                 quickSort_2511533016(arr_3016, low_3016,
52.             pi_3016 - 1);
53.                 quickSort_2511533016(arr_3016, pi_3016 + 1,
54.             high_3016);
55.             }
56.         }
57.         public static void printArr_2511533016(int[] arr_3016) {
58.             for (int i_3016 = 0; i_3016 < arr_3016.length;
59.                 i_3016++) {
60.                 System.out.print(arr_3016[i_3016] + " ");
61.             }
62.             System.out.println();
63.         }
64.         public static void main(String[] args) {
65.             int[] arr_3016 = { 10, 7, 8, 9, 1, 5};
66.             int N_3016 = arr_3016.length;
67.             System.out.print("Data sebelum diurutkan : ");
68.             printArr_2511533016(arr_3016);
69.             quickSort_2511533016(arr_3016, 0, N_3016 - 1);
70.             System.out.print("Data terurut quicksort : ");
71.             printArr_2511533016(arr_3016);
72.         }
73.     }

```

```

Data sebelum diurutkan : 10 7 8 9 1 5
Data terurut quicksort : 1 5 7 8 9 10

```

Gambar 2.2.2

Kode di atas adalah implementasi algoritma Quick Sort yang berada dalam package `Pekan8_2511533016`. Kelas utamanya bernama `QuickSort_2511533016` yang berisi empat fungsi pendukung dan satu fungsi main untuk menjalankan program secara keseluruhan.

Fungsi pertama adalah `swap_2511533016` yang bertugas menukar posisi dua elemen dalam array menggunakan variabel sementara `temp_3016`. Fungsi ini digunakan berulang kali oleh fungsi-fungsi lainnya setiap kali ada dua elemen yang perlu ditukar posisinya.

Fungsi kedua adalah `medianOfThree_2511533016` yang merupakan keunggulan utama dari program ini dibanding Quick Sort biasa. Fungsi ini bekerja dengan membandingkan tiga elemen yaitu elemen paling kiri `low`, elemen tengah `mid`, dan elemen paling kanan `high`, kemudian mengurutkan ketiganya sehingga nilai median atau nilai tengah dari ketiganya akan berada di posisi `mid`. Setelah itu nilai median dipindahkan ke posisi `high` agar bisa digunakan sebagai pivot. Tujuan dari pendekatan ini adalah untuk menghindari kondisi terburuk Quick Sort yang bisa terjadi jika pivot yang dipilih selalu merupakan nilai terkecil atau terbesar dari data.

Fungsi ketiga adalah `partition_2511533016` yang bertugas membagi array menjadi dua bagian berdasarkan nilai pivot. Fungsi ini pertama memanggil `medianOfThree_2511533016` untuk menentukan pivot yang optimal, kemudian menggunakan nilai `arr[high]` sebagai pivot. Perulangan `for` berjalan dari indeks `low` hingga `high - 1` untuk membandingkan setiap elemen dengan pivot. Jika elemen lebih kecil dari pivot maka elemen tersebut ditukar ke sisi kiri, dan setelah perulangan selesai pivot ditempatkan di posisi yang tepat di antara dua kelompok tersebut. Fungsi ini mengembalikan indeks posisi pivot sebagai hasil partisi.

Fungsi keempat adalah `quickSort_2511533016` yang merupakan fungsi rekursif utama. Fungsi ini memanggil `partition_2511533016` untuk mendapatkan posisi pivot, kemudian memanggil dirinya sendiri secara rekursif untuk bagian kiri yaitu elemen-elemen sebelum pivot, dan bagian kanan yaitu elemen-elemen setelah pivot. Proses ini terus berulang hingga setiap bagian hanya memiliki satu elemen dan tidak bisa dibagi lagi.

Fungsi `printArr_2511533016` adalah fungsi pembantu yang bertugas mencetak seluruh isi array ke layar. Pada fungsi `main`, program mendefinisikan array dengan enam elemen yaitu `{10, 7, 8, 9, 1, 5}` yang belum terurut. Program mencetak array awal, kemudian memanggil `quickSort_2511533016` untuk mengurutkannya, dan terakhir mencetak hasil array yang sudah terurut menjadi `1 5 7 8 9 10`.

Kesimpulan: Kode program ini merupakan implementasi algoritma Quick Sort dengan metode Median-of-Three dalam bahasa Java, yang mendemonstrasikan cara mengurutkan sekumpulan data integer dari nilai terkecil ke terbesar (ascending) menggunakan pendekatan divide and conquer dengan pemilihan pivot yang lebih optimal dibandingkan Quick Sort biasa sehingga menghasilkan performa pengurutan yang lebih baik.

2.2.3 Class Insertion sort (InsertionSort_2511533016)

```

1. package Pekan8_2511533016;
2.
3. public class ShellSort_2511533016 {
4.     public static void ShellSort(int[] A) {
5.         int n_3016 = A.length;
6.         int gap_3016 = n_3016 / 2;
7.         while (gap_3016 > 0) {
8.             for (int i_3016 = gap_3016; i_3016 < n_3016;
9.                 i_3016++) {
10.                 int temp_3016 = A[i_3016];
11.                 int j_3016 = i_3016;
12.                 while (j_3016 >= gap_3016 && A[j_3016 -
13.                     gap_3016] > temp_3016) {
14.                         A[j_3016] = A[j_3016 -
15.                             gap_3016];
16.                         j_3016 = j_3016 - gap_3016;
17.                     }
18.                     A[j_3016] = temp_3016;
19.                 }
20.                 gap_3016 = gap_3016 / 2;
21.             }
22.         }
23.     }
24.     public static void main (String[] args) {
25.         int[] data_3016 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
26.
27.         System.out.print("sebelum : ");
28.         printArray(data_3016);
29.
30.         ShellSort(data_3016);

```

```

28.
29.         System.out.println("Sesudah (shell sort) : ");
30.         printArray(data_3016);
31.     }
32.
33.     public static void printArray(int[] arr) {
34.         for (int i_3016 : arr) System.out.print(i_3016 + "
35.         ");
36.         System.out.println();
37.     }

```

```

sebelum : 3 10 4 6 8 9 7 2 1 5
Sesudah (shell sort) :
1 2 3 4 5 6 7 8 9 10

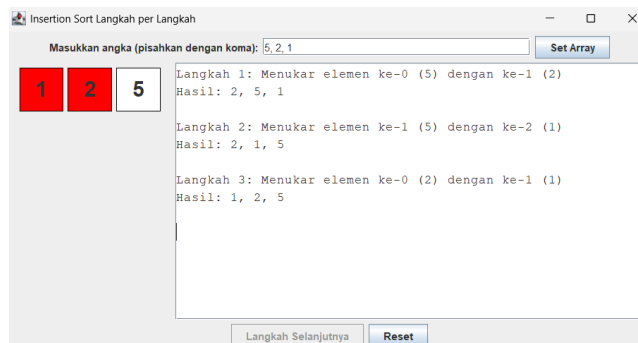
```

Gambar 2.2.3

Kode di atas adalah implementasi algoritma Shell Sort yang berada dalam package `Pekan8_2511533016`. Kelas utamanya bernama `ShellSort_2511533016` yang berisi fungsi utama pengurutan, fungsi pembantu untuk mencetak array, dan fungsi main untuk menjalankan program.

Fungsi `ShellSort` menerima parameter berupa array integer yang akan diurutkan. Pertama program menghitung nilai awal `gap_3016` dengan cara membagi panjang array dengan 2. Nilai `gap` ini menentukan jarak antar elemen yang akan dibandingkan dan ditukar. Perulangan `while` terluar akan terus berjalan selama nilai `gap` masih lebih besar dari nol, dan setiap kali perulangan ini selesai satu putaran maka nilai `gap` dibagi dua lagi sehingga jarak perbandingan semakin dipersempit dari putaran ke putaran.

Di dalam perulangan `while` terluar terdapat perulangan `for` yang berjalan dari indeks `gap_3016` hingga akhir array. Pada setiap iterasi, elemen di posisi `i_3016` disimpan ke variabel sementara `temp_3016`. Kemudian perulangan `while` dalam berjalan selama indeks `j_3016` masih valid dan elemen yang berada sejauh `gap` di belakangnya lebih besar dari `temp_3016`. Selama kondisi itu terpenuhi, elemen tersebut digeser ke kanan sejauh satu `gap`, dan indeks `j_3016` dikurangi sebesar nilai `gap`. Setelah perulangan dalam selesai, nilai `temp_3016` disisipkan ke posisi `j_3016` yang telah ditemukan. Cara kerja ini mirip dengan Insertion Sort, hanya saja perbandingan dan pergeseran dilakukan pada elemen-elemen yang berjarak `gap`, bukan elemen yang berdekatan langsung.

[illegible]

Gambar 2.2.4

Kode di atas adalah implementasi Bubble Sort dengan tampilan grafis yang berada dalam package `Pekan8_2511533016`. Kelas utamanya bernama `BubbleSortGUI_2511533016` yang mewarisi kelas `JFrame` dari library Java Swing. Di dalam kelas terdapat beberapa atribut utama yaitu array untuk menyimpan data angka, array label untuk menampilkan setiap elemen sebagai kotak visual, tombol-tombol kontrol, field input, serta variabel `i_3016` sebagai penanda pass dan `j_3016` sebagai penanda posisi perbandingan pada setiap langkah sorting.

Pada bagian constructor, program membangun tampilan jendela berukuran 750x400 piksel dengan layout BorderLayout. Panel input di bagian atas berisi field teks untuk memasukkan angka yang dipisahkan koma dan tombol Set Array. Panel tengah menampilkan elemen-elemen array dalam bentuk kotak berlabel. Panel bawah berisi tombol Langkah Selanjutnya dan Reset, sedangkan di bagian kanan terdapat area teks bercakupan scroll yang menampilkan log setiap langkah pengurutan secara detail.

Fungsi `setArrayFromInput_3016` bertugas membaca dan memvalidasi input angka dari field teks, kemudian menginisialisasi array dan menampilkan setiap elemennya sebagai kotak berlabel berwarna putih di panel tengah. Jika input mengandung karakter selain angka dan koma maka program menampilkan pesan error melalui `JOptionPane`. Variabel `i_3016` dan `j_3016` direset ke nol sebagai tanda proses sorting siap dimulai dari awal.

Fungsi `performStep_3016` adalah inti dari program ini karena menjalankan tepat satu langkah Bubble Sort setiap kali tombol ditekan. Di setiap langkah, dua elemen yang sedang dibandingkan yaitu di posisi `j_3016` dan `j_3016 + 1` diberi warna biru muda sebagai penanda sedang diproses. Jika elemen kiri lebih besar dari elemen kanan maka keduanya ditukar menggunakan variabel sementara `temp_3016` dan warnanya berubah menjadi merah sebagai tanda pertukaran terjadi. Jika tidak ada pertukaran, warna tetap biru muda dan log mencatat bahwa tidak ada pertukaran pada langkah tersebut. Setiap langkah dicatat lengkap di area log beserta kondisi array setelahnya. Setelah setiap langkah selesai, `j_3016` ditambah satu untuk berpindah ke pasangan berikutnya. Jika `j_3016` sudah mencapai batas pass saat ini maka `j_3016` direset ke nol dan `i_3016` ditambah satu untuk memulai pass berikutnya. Ketika `i_3016` sudah mencapai panjang array dikurangi satu maka seluruh proses sorting dinyatakan selesai dan tombol langkah dinonaktifkan.

Fungsi-fungsi pendukung lainnya adalah `updateLabels_3016` untuk memperbarui teks kotak setelah pertukaran terjadi, `resetHighlights_3016` untuk mengembalikan warna semua kotak ke putih sebelum langkah baru dimulai, `reset_3016` untuk mengembalikan seluruh tampilan dan variabel ke kondisi awal, serta `arrayToString_3016` untuk mengubah isi array menjadi teks yang

ditampilkan di area log. Pada fungsi main, program dijalankan menggunakan `SwingUtilities.invokeLater` agar tampilan GUI dibuat di thread yang benar sesuai standar pemrograman Java Swing.

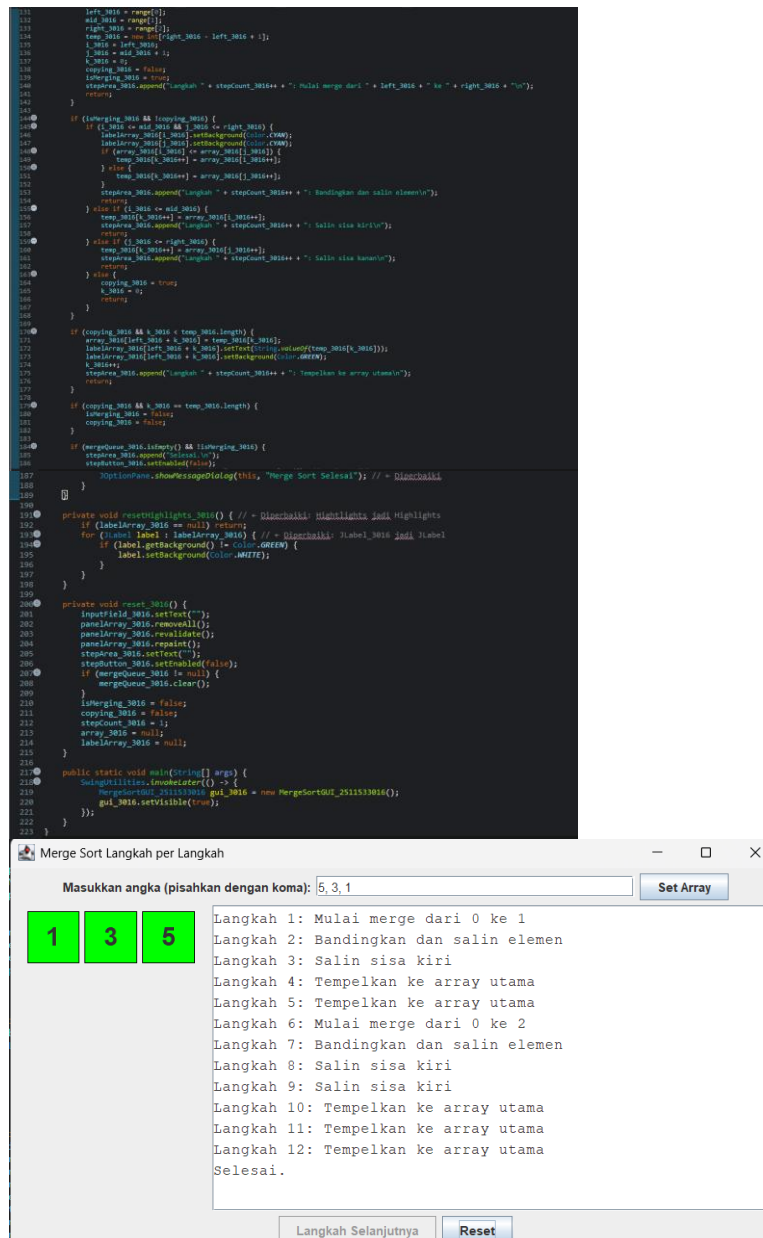
Kesimpulan: Kode program ini merupakan implementasi algoritma Bubble Sort berbasis antarmuka grafis (GUI) menggunakan Java Swing, yang mendemonstrasikan proses pengurutan data secara visual dan interaktif langkah per langkah dengan memberikan highlight warna biru muda pada elemen yang sedang dibandingkan dan warna merah pada elemen yang sedang ditukar, sehingga pengguna dapat mengikuti dan memahami alur proses Bubble Sort secara langsung di setiap tahapannya.

2.2.5 Class Merge Sort GUI (MergeSortGUI_2511533016)

```

1 package Pkani_2511533016;
2 import java.awt.BorderLayout;
3
4 public class MergeSortGUI_2511533016 extends JFrame {
5     private static final long serialVersionUID = 1L;
6     private int[] array;
7     private JButton[] btnArray;
8     private JButton stepButton, resetButton, setButton;
9     private JTextField inputField;
10    private JPanel panelArray;
11    private JTextArea stepArea;
12
13    private Queue<int> mergeQueue;
14    private boolean isMerging;
15    private boolean copying;
16    private int left, mid, right;
17    private int[] temp;
18    private int i, j, k;
19    private int stepCount;
20
21    public MergeSortGUI_2511533016() {
22        setTitle("Merge Sort Langkah per Langkah");
23        setSize(700, 400);
24        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25        setLocationRelativeTo(null);
26        setLayout(new BorderLayout());
27
28        JPanel inputPanel = new JPanel(new FlowLayout());
29        inputField = new JTextField(40);
30        btnArray = new JButton[10];
31        setButton = new JButton("Set Array");
32        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma)"));
33        inputPanel.add(inputField);
34        inputPanel.add(setButton);
35
36        panelArray = new JPanel();
37        panelArray.setLayout(new FlowLayout());
38
39        JPanel controlPanel = new JPanel();
40        stepButton = new JButton("Langkah Selanjutnya");
41        resetButton = new JButton("Reset");
42        stepButton.setEnabled(false);
43        controlPanel.add(stepButton);
44        controlPanel.add(resetButton);
45
46        stepArea = new JTextArea(8, 60);
47        stepArea.setEditable(false);
48        JScrollPane scrollPane = new JScrollPane(stepArea);
49
50        add(inputPanel, BorderLayout.NORTH);
51        add(panelArray, BorderLayout.CENTER);
52        add(controlPanel, BorderLayout.SOUTH);
53        add(scrollPane, BorderLayout.EAST);
54
55        stepButton.addActionListener(e -> nextStep());
56        resetButton.addActionListener(e -> reset());
57        setButton.addActionListener(e -> setArray());
58
59        private void generateRandomArray(int n) {
60            int i;
61            for (i = 0; i < n; i++) {
62                array[i] = (int) (Math.random() * 100);
63            }
64        }
65
66        private void setArray() {
67            String text = inputField.getText().trim();
68            if (text.isEmpty()) return;
69            String[] parts = text.split(",");
70            array = new int[parts.length];
71            for (int i = 0; i < parts.length; i++) {
72                array[i] = Integer.parseInt(parts[i].trim());
73            }
74        }
75
76        private void nextStep() {
77            if (array.length == 0) return;
78            if (isMerging) {
79                // Merging process
80                // ... (omitted code for merging) ...
81            } else {
82                // Splitting process
83                // ... (omitted code for splitting) ...
84            }
85            stepCount++;
86            stepArea.append("Step " + stepCount + ": " + arrayToString(array) + "\n");
87            scrollPane.getViewport().repaint();
88        }
89
90        private String arrayToString(int[] arr) {
91            StringBuilder sb = new StringBuilder();
92            for (int i = 0; i < arr.length; i++) {
93                sb.append(arr[i] + " ");
94            }
95            return sb.toString().trim();
96        }
97
98        public static void main(String[] args) {
99            SwingUtilities.invokeLater(() -> {
100                MergeSortGUI_2511533016 gui = new MergeSortGUI_2511533016();
101                gui.setVisible(true);
102            });
103        }
104    }

```



Gambar 2.2.5

Kode di atas adalah implementasi Merge Sort dengan tampilan grafis yang berada dalam package `Pekan8_2511533016`. Kelas utamanya bernama `MergeSortGUI_2511533016` yang mewarisi kelas `JFrame` dari library Java Swing. Di dalam kelas terdapat beberapa atribut penting yaitu array untuk menyimpan data, array label untuk menampilkan elemen sebagai kotak visual, tombol-tombol kontrol, field input, area log, serta beberapa variabel khusus seperti `mergeQueue_3016` bertipe `Queue`, `isMerging_3016`, `copying_3016`, dan variabel indeks `i_3016`, `j_3016`, `k_3016` yang semuanya berperan dalam mengatur jalannya proses merge secara bertahap.

Pada bagian constructor, program membangun tampilan jendela berukuran 750x400 piksel menggunakan BorderLayout. Panel input di bagian atas berisi field teks untuk memasukkan angka yang dipisahkan koma dan tombol Set Array. Panel tengah menampilkan elemen-elemen array dalam bentuk kotak berlabel. Panel bawah berisi tombol Langkah Selanjutnya dan Reset, sedangkan di bagian kanan terdapat area teks bercakupan scroll untuk menampilkan log setiap langkah. Fungsi `generateMergeSteps_3016` adalah fungsi rekursif yang bekerja di balik layar sebelum proses visualisasi dimulai. Fungsi ini menelusuri seluruh proses pembagian array secara rekursif dari kiri ke kanan, kemudian menyimpan setiap operasi penggabungan yang perlu dilakukan ke dalam `mergeQueue_3016` dalam urutan yang benar. Dengan pendekatan ini semua langkah merge sudah disiapkan terlebih dahulu sehingga proses visualisasi bisa dilakukan satu per satu sesuai antrian.

Fungsi `setArrayFromInput_3016` bertugas membaca dan memvalidasi input angka dari field teks, menginisialisasi array dan label kotak visual, lalu memanggil `generateMergeSteps_3016` untuk mengisi antrian langkah merge. Setelah antrian siap, tombol Langkah Selanjutnya diaktifkan dan proses visualisasi siap dimulai.

Fungsi `performStep_3016` adalah inti dari program ini dan bekerja dalam tiga fase berbeda. Fase pertama terjadi ketika `isMerging_3016` bernilai false, program mengambil satu operasi merge dari antrian dan menginisialisasi variabel-variabel yang dibutuhkan untuk proses penggabungan tersebut. Fase kedua terjadi ketika `isMerging_3016` bernilai true dan `copying_3016` bernilai false, di sinilah proses perbandingan dan pemilihan elemen terkecil dari dua bagian array berlangsung. Elemen yang sedang dibandingkan diberi warna biru muda, dan elemen yang terpilih disalin ke array sementara `temp_3016`. Jika salah satu sisi sudah habis maka sisa elemen dari sisi yang masih ada langsung disalin. Fase ketiga terjadi ketika `copying_3016` bernilai true, di mana isi array sementara disalin kembali ke array utama satu per satu dan setiap elemen yang berhasil ditempatkan diberi warna hijau sebagai tanda sudah terurut. Setelah semua antrian habis dan tidak ada proses merge yang berjalan, program menyatakan sorting selesai.

Fungsi `resetHighlights_3016` bertugas mengembalikan warna kotak ke putih sebelum langkah berikutnya dimulai, namun kotak yang sudah berwarna hijau tidak ikut direset karena menandakan elemen yang sudah berada di posisi akhirnya. Fungsi `reset_3016` membersihkan seluruh tampilan dan variabel ke kondisi awal. Pada fungsi `main`, program dijalankan menggunakan `SwingUtilities.invokeLater` agar tampilan GUI dibuat di thread yang benar sesuai standar pemrograman Java Swing.

Kesimpulan: Kode program ini merupakan implementasi algoritma Merge Sort berbasis antarmuka grafis (GUI) menggunakan Java Swing, yang mendemonstrasikan proses pengurutan data secara visual dan interaktif langkah per langkah dengan memberikan highlight warna biru muda pada elemen yang sedang dibandingkan dan warna hijau pada elemen yang sudah berhasil digabungkan ke posisi yang benar, sehingga pengguna dapat memahami alur divide and conquer dari Merge Sort secara bertahap dan terstruktur.

BAB III

PENUTUP

3.1 Kesimpulan

Materi sorting merupakan salah satu konsep dasar dalam struktur data dan algoritma yang digunakan untuk mengurutkan data berdasarkan aturan tertentu, seperti urutan alfabet atau angka. Pada program di atas, proses sorting diterapkan pada data mahasiswa menggunakan tiga metode pengurutan yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Ketiga metode tersebut memiliki cara kerja yang berbeda, namun memiliki tujuan yang sama yaitu menyusun data agar lebih teratur dan mudah dicari maupun diproses.

Program yang dibuat menggunakan Java Swing berhasil menampilkan visualisasi proses sorting secara bertahap melalui antarmuka GUI. Pengguna dapat memasukkan data mahasiswa berupa nama, NIM, dan program studi, kemudian memilih metode sorting yang diinginkan. Setiap langkah pengurutan ditampilkan secara visual sehingga pengguna dapat memahami bagaimana proses perpindahan dan pertukaran data terjadi pada setiap algoritma sorting. Selain itu, program juga menerapkan konsep Object Oriented Programming (OOP) melalui penggunaan class Mahasiswa_2511533016 sebagai model penyimpanan data mahasiswa.

Secara keseluruhan, program ini menunjukkan bahwa visualisasi algoritma dapat membantu dalam memahami konsep sorting dengan lebih mudah, interaktif, dan terstruktur. Program juga membuktikan bahwa Java Swing dapat digunakan untuk membangun aplikasi edukasi sederhana yang mampu menggabungkan konsep GUI, struktur data, algoritma sorting, dan OOP dalam satu sistem.

3.2 Saran

Program dapat dikembangkan dengan menambahkan animasi visual sorting, penggunaan JTable agar tampilan lebih rapi, serta penambahan fitur sorting berdasarkan NIM atau program studi. Selain itu, dapat digunakan algoritma sorting yang lebih cepat seperti Quick Sort atau Merge Sort untuk data yang lebih besar.

DAFTAR PUSTAKA

Astuti, W., & Prabowo, H. (2020). Implementasi Algoritma Sorting pada Struktur Data dalam Bahasa Pemrograman Java. *Jurnal Informatika dan Sistem Informasi*, 6(2), 112–125.

LAPORAN TUGAS PRAKTIKUM

SOAL :

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A Z) atau Durasi (terpendek ke terpanjang).

1. Class Lagu (Lagu_2511533016)

```
1. package Pekan8_2511533016;
2.
3. public class Lagu_2511533016 {
4.
5.     String judul_3016;
6.     String penyanyi_3016;
7.     int durasi_3016;
8.
9.     public Lagu_2511533016(String judul_3016,String
    penyanyi_3016,int durasi_3016) {
10.
11.         this.judul_3016 = judul_3016;
12.         this.penyanyi_3016 = penyanyi_3016;
13.         this.durasi_3016 = durasi_3016;
14.     }
15. }
```

Gambar 1

Program di atas merupakan sebuah kelas Java bernama Lagu_2511533016 yang digunakan sebagai representasi atau model data untuk menyimpan informasi sebuah lagu. Kelas ini berada dalam package Pekan8_2511533016 dan memiliki tiga atribut utama, yaitu judul_3016 yang bertipe data String untuk menyimpan judul lagu, penyanyi_3016 yang bertipe data String untuk menyimpan nama penyanyi, serta durasi_3016 yang bertipe data int untuk menyimpan durasi lagu dalam satuan detik.

Di dalam kelas terdapat sebuah constructor Lagu_2511533016(String judul_3016, String penyanyi_3016, int durasi_3016) yang berfungsi untuk menginisialisasi nilai dari setiap atribut saat objek lagu dibuat. Penggunaan kata kunci this digunakan untuk membedakan atribut milik kelas dengan parameter

yang diterima oleh constructor. Dengan adanya constructor ini, setiap objek lagu yang dibuat dapat langsung memiliki data judul, penyanyi, dan durasi sesuai nilai yang diberikan saat proses pembuatan objek berlangsung.

Kelas ini berperan sebagai struktur data yang digunakan oleh program utama untuk menyimpan dan mengelola informasi lagu dalam sebuah playlist. Setiap objek yang dibuat dari kelas ini akan mewakili satu lagu yang nantinya dapat ditampilkan, disimpan dalam array, maupun diurutkan menggunakan berbagai algoritma sorting seperti Shell Sort, Quick Sort, dan Merge Sort.

Kesimpulan : Program di atas menunjukkan implementasi sebuah kelas objek (class) dalam konsep Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP) yang berfungsi sebagai wadah penyimpanan data lagu. Kelas ini digunakan untuk merepresentasikan karakteristik sebuah lagu melalui atribut judul, penyanyi, dan durasi, serta menyediakan constructor untuk menginisialisasi data tersebut. Dengan demikian, kelas Lagu_2511533016 berperan sebagai model data yang menjadi dasar dalam pengelolaan playlist dan proses pengurutan lagu pada program utama.

2. Class Sorting (Sorting_2511533016)

```

1. package Pekan8_2511533016;
2.
3. import java.util.Scanner;
4.
5. public class Sorting_2511533016 {
6.     Lagu_2511533016[] dataLagu_3016 = new Lagu_2511533016[20];
7.     int jumlahData_3016 = 0;
8.
9.     // PLAYLIST LAGU
10.    public void inputData_3016() {
11.        dataLagu_3016[jumlahData_3016++] = //Song 1
12.            new Lagu_2511533016("Aku Pasti
13.            Kembali", "Pasto", 440);
14.        dataLagu_3016[jumlahData_3016++] = //Song 2
15.            new Lagu_2511533016("Sing To You",
16.            "Silodinasti", 233);
17.        dataLagu_3016[jumlahData_3016++] = //Song 3
18.            new Lagu_2511533016("Be For You Go",
19.            "Lewis Capaldi", 334);
20.        dataLagu_3016[jumlahData_3016++] = //Song 4
21.            new Lagu_2511533016("Poor Grammer",
22.            "Roar", 225);

```

```

19.         dataLagu_3016[jumlahData_3016++] = //Song 5
20.             new Lagu_2511533016("Kursi Goyang",
    "Fourtwnty", 257);
21.         dataLagu_3016[jumlahData_3016++] = //Song 6
22.             new Lagu_2511533016("Mari Bercerita",
    "Payung Teduh", 241);
23.         dataLagu_3016[jumlahData_3016++] = //Song 7
24.             new Lagu_2511533016("Bimbang", "Melly
    Goeslaw", 226);
25.         dataLagu_3016[jumlahData_3016++] = //Song 8
26.             new Lagu_2511533016("Hysteria", "Muse",
    344);
27.         dataLagu_3016[jumlahData_3016++] = //Song 9
28.             new Lagu_2511533016("Ode To the Mets",
    "The Strokes", 288);
29.         dataLagu_3016[jumlahData_3016++] = //Song 10
30.             new Lagu_2511533016("Young",
    "Vacations", 271);
31.     }
32.
33.     // MENAMPILKAN DATA
34.     public void tampilData_3016() {
35.         for (int i_3016 = 0; i_3016 < jumlahData_3016;
    i_3016++) {
36.             System.out.println(
37.                 (i_3016 + 1) + ". "
38.                 +
    dataLagu_3016[i_3016].judul_3016
39.                 + " - "
40.                 +
    dataLagu_3016[i_3016].penyanyi_3016
41.                 + " - "
42.                 +
    dataLagu_3016[i_3016].durasi_3016
43.                 + " detik");
44.         }
45.     }
46.
47.     // ALGORTIMA SHELL SORT (JUDUL A-Z)
48.     public void shellSort_3016() {
49.         for (int gap_3016 = jumlahData_3016 / 2; gap_3016 >
    0; gap_3016 /= 2) {
50.             for (int i_3016 = gap_3016; i_3016 <
    jumlahData_3016; i_3016++) {
51.                 Lagu_2511533016 temp_3016 =
    dataLagu_3016[i_3016];
52.                 int j_3016;
53.                 for (j_3016 = i_3016;
54.                     j_3016 >= gap_3016
55.                     &&
    dataLagu_3016[j_3016 - gap_3016]
56.                     .judul_3016.compareToIgnoreCase(
57.                         temp_3016.judul_3016) > 0;
58.                     j_3016 -= gap_3016) {

```

```

59.         dataLagu_3016[j_3016] =
dataLagu_3016[j_3016 - gap_3016];
60.     }
61.         dataLagu_3016[j_3016] = temp_3016;
62.     }
63. }
64. }
65.
66. // ALGORITMA QUICK SORT (DURASI ASC)
67. public void quickSort_3016(int low_3016, int high_3016) {
68.     if (low_3016 < high_3016) {
69.         int pi_3016 = partition_3016(low_3016,
high_3016);
70.         quickSort_3016(low_3016, pi_3016 - 1);
71.         quickSort_3016(pi_3016 + 1, high_3016);
72.     }
73. }
74. public int partition_3016(int low_3016, int high_3016) {
75.     int pivot_3016 =
dataLagu_3016[high_3016].durasi_3016;
76.     int i_3016 = low_3016 - 1;
77.     for (int j_3016 = low_3016; j_3016 < high_3016;
j_3016++) {
78.         if (dataLagu_3016[j_3016].durasi_3016 <=
pivot_3016) {
79.             i_3016++;
80.             Lagu_2511533016 temp_3016 =
dataLagu_3016[i_3016];
81.             dataLagu_3016[i_3016] =
dataLagu_3016[j_3016];
82.             dataLagu_3016[j_3016] = temp_3016;
83.         }
84.     }
85.     Lagu_2511533016 temp_3016 = dataLagu_3016[i_3016 +
1];
86.     dataLagu_3016[i_3016 + 1] =
dataLagu_3016[high_3016];
87.     dataLagu_3016[high_3016] = temp_3016;
88.     return i_3016 + 1;
89. }
90.
91. // ALGORITMA MERGE SORT (JUDUL A-Z)
92. public void mergeSort_3016(int kiri_3016, int kanan_3016) {
93.     if (kiri_3016 < kanan_3016) {
94.         int tengah_3016 = (kiri_3016 + kanan_3016) /
2;
95.         mergeSort_3016(kiri_3016, tengah_3016);
96.         mergeSort_3016(tengah_3016 + 1,
kanan_3016);
97.         merge_3016(kiri_3016, tengah_3016,
kanan_3016);
98.     }
99. }

```

```

104.         }
105.         public void merge_3016(int kiri_3016, int
    tengah_3016, int kanan_3016) {
106.             int n1_3016 = tengah_3016 - kiri_3016 + 1;
107.             int n2_3016 = kanan_3016 - tengah_3016;
108.
109.             Lagu_2511533016[] kiriArray_3016 = new
    Lagu_2511533016[n1_3016];
110.             Lagu_2511533016[] kananArray_3016 = new
    Lagu_2511533016[n2_3016];
111.
112.             for (int i_3016 = 0; i_3016 < n1_3016;
    i_3016++)
113.                 kiriArray_3016[i_3016] =
    dataLagu_3016[kiri_3016 + i_3016];
114.             for (int j_3016 = 0; j_3016 < n2_3016;
    j_3016++)
115.                 kananArray_3016[j_3016] =
    dataLagu_3016[tengah_3016 + 1 + j_3016];
116.
117.             int i_3016 = 0;
118.             int j_3016 = 0;
119.             int k_3016 = kiri_3016;
120.
121.             while (i_3016 < n1_3016 && j_3016 < n2_3016)
    {
122.                 if
    (kiriArray_3016[i_3016].judul_3016.compareToIgnoreCase(
123.                 kananArray_3016[j_3016].judul_3016) <= 0) {
124.                     dataLagu_3016[k_3016] =
    kiriArray_3016[i_3016];
125.                     i_3016++;
126.                 } else {
127.                     dataLagu_3016[k_3016] =
    kananArray_3016[j_3016];
128.                     j_3016++;
129.                 }
130.                 k_3016++;
131.             }
132.
133.             while (i_3016 < n1_3016) {
134.                 dataLagu_3016[k_3016] =
    kiriArray_3016[i_3016];
135.                 i_3016++;
136.                 k_3016++;
137.             }
138.
139.             while (j_3016 < n2_3016) {
140.                 dataLagu_3016[k_3016] =
    kananArray_3016[j_3016];
141.                 j_3016++;
142.                 k_3016++;
143.             }
144.         }
145.
146.         // MAIN

```

```

147.         public static void main(String[] args) {
148.             Scanner input_3016 = new Scanner(System.in);
149.             Sorting_2511533016 playlist_3016 = new
Sorting_2511533016();
150.             playlist_3016.inputData_3016();
151.
152.             System.out.println("=== Sorting Playlist NIM:
2511533016 ===");
153.             System.out.print("Pilih Algoritma (1 = Shell,
2 = Quick, 3 = Merge): ");
154.             int pilihan_3016 = input_3016.nextInt();
155.
156.             if (pilihan_3016 < 1 || pilihan_3016 > 3) {
157.                 System.out.println();
158.                 System.out.println("Pilihan tidak
valid.");
159.                 input_3016.close();
160.                 return;
161.             }
162.
163.             System.out.println("\nData Sebelum
Sorting:");
164.             playlist_3016.tampilData_3016();
165.
166.             switch (pilihan_3016) {
167.                 case 1:
168.                     playlist_3016.shellSort_3016();
169.                     System.out.println("\nData
Setelah Shell Sort (Judul A-Z):");
170.                     break;
171.                 case 2:
172.                     playlist_3016.quickSort_3016(0,
playlist_3016.jumlahData_3016 - 1);
173.                     System.out.println("\nData
Setelah Quick Sort (Durasi Asc):");
174.                     break;
175.                 case 3:
176.                     playlist_3016.mergeSort_3016(0,
playlist_3016.jumlahData_3016 - 1);
177.                     System.out.println("\nData
Setelah Merge Sort (Judul A-Z):");
178.                     break;
179.             }
180.             playlist_3016.tampilData_3016();
181.             input_3016.close();
182.         }
183.     }

```

Gambar 2

Program di atas merupakan program Java yang digunakan untuk mengelola dan mengurutkan data playlist lagu menggunakan beberapa algoritma sorting. Program ini menggunakan sebuah array objek Lagu_2511533016 untuk menyimpan data lagu yang terdiri dari judul lagu, nama penyanyi, dan durasi lagu dalam satuan detik. Pada method

inputData_3016(), program mengisi data playlist dengan 10 lagu yang telah ditentukan sebelumnya. Method tampilData_3016() berfungsi untuk menampilkan seluruh data lagu yang tersimpan dalam playlist ke layar.

Program menyediakan tiga metode pengurutan yang dapat dipilih oleh pengguna. Metode pertama adalah **Shell Sort** yang diimplementasikan pada method shellSort_3016(), digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis dari A sampai Z. Metode kedua adalah **Quick Sort** yang terdiri dari method quickSort_3016() dan partition_3016(), digunakan untuk mengurutkan lagu berdasarkan durasi secara ascending (dari durasi terpendek ke terpanjang). Metode ketiga adalah **Merge Sort** yang terdiri dari method mergeSort_3016() dan merge_3016(), digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis dengan teknik pembagian data menjadi beberapa bagian kecil kemudian digabungkan kembali dalam keadaan terurut.

Pada bagian main(), program pertama kali membuat objek playlist dan mengisi data lagu yang tersedia. Selanjutnya pengguna diminta memilih algoritma sorting yang ingin digunakan, yaitu Shell Sort, Quick Sort, atau Merge Sort. Program kemudian menampilkan data sebelum proses pengurutan dilakukan, menjalankan algoritma yang dipilih, dan akhirnya menampilkan hasil data setelah proses sorting selesai. Dengan demikian, pengguna dapat membandingkan kondisi data sebelum dan sesudah dilakukan pengurutan.

Kesimpulan: Program ini menunjukkan implementasi dan perbandingan penggunaan tiga algoritma pengurutan, yaitu Shell Sort, Quick Sort, dan Merge Sort, pada data playlist lagu. Melalui program ini, pengguna dapat memahami cara kerja masing-masing algoritma dalam mengurutkan data berdasarkan kriteria tertentu, seperti judul lagu dan durasi lagu. Selain itu, program juga menunjukkan penerapan konsep array objek, pemrograman berorientasi objek (OOP), rekursi, serta algoritma sorting dalam bahasa Java untuk mengelola dan mengolah data secara lebih terstruktur dan efisien.

OUTPUT :

SHELL SORT :

```
=== Sorting Playlist NIM: 2511533016 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 1

Data Sebelum Sorting:
1. Aku Pasti Kembali - Pasto - 440 detik
2. Sing To You - Silodinasti - 233 detik
3. Be For You Go - Lewis Capaldi - 334 detik
4. Poor Grammer - Roar - 225 detik
5. Kursi Goyang - Fourtwny - 257 detik
6. Mari Bercerita - Payung Teduh - 241 detik
7. Bimbang - Melly Goeslaw - 226 detik
8. Hysteria - Muse - 344 detik
9. Ode To the Mets - The Strokes - 288 detik
10. Young - Vacations - 271 detik

Data Setelah Shell Sort (Judul A-Z):
1. Aku Pasti Kembali - Pasto - 440 detik
2. Be For You Go - Lewis Capaldi - 334 detik
3. Bimbang - Melly Goeslaw - 226 detik
4. Hysteria - Muse - 344 detik
5. Kursi Goyang - Fourtwny - 257 detik
6. Mari Bercerita - Payung Teduh - 241 detik
7. Ode To the Mets - The Strokes - 288 detik
8. Poor Grammer - Roar - 225 detik
9. Sing To You - Silodinasti - 233 detik
10. Young - Vacations - 271 detik
```

MERGE SORT :

```
=== Sorting Playlist NIM: 2511533016 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 3

Data Sebelum Sorting:
1. Aku Pasti Kembali - Pasto - 440 detik
2. Sing To You - Silodinasti - 233 detik
3. Be For You Go - Lewis Capaldi - 334 detik
4. Poor Grammer - Roar - 225 detik
5. Kursi Goyang - Fourtwny - 257 detik
6. Mari Bercerita - Payung Teduh - 241 detik
7. Bimbang - Melly Goeslaw - 226 detik
8. Hysteria - Muse - 344 detik
9. Ode To the Mets - The Strokes - 288 detik
10. Young - Vacations - 271 detik

Data Setelah Merge Sort (Judul A-Z):
1. Aku Pasti Kembali - Pasto - 440 detik
2. Be For You Go - Lewis Capaldi - 334 detik
3. Bimbang - Melly Goeslaw - 226 detik
4. Hysteria - Muse - 344 detik
5. Kursi Goyang - Fourtwny - 257 detik
6. Mari Bercerita - Payung Teduh - 241 detik
7. Ode To the Mets - The Strokes - 288 detik
8. Poor Grammer - Roar - 225 detik
9. Sing To You - Silodinasti - 233 detik
10. Young - Vacations - 271 detik
```

QUICK SORT :

```
=== Sorting Playlist NIM: 2511533016 ===
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 2

Data Sebelum Sorting:
1. Aku Pasti Kembali - Pasto - 440 detik
2. Sing To You - Silodinasti - 233 detik
3. Be For You Go - Lewis Capaldi - 334 detik
4. Poor Grammer - Roar - 225 detik
5. Kursi Goyang - Fourtwny - 257 detik
6. Mari Bercerita - Payung Teduh - 241 detik
7. Bimbang - Melly Goeslaw - 226 detik
8. Hysteria - Muse - 344 detik
9. Ode To the Mets - The Strokes - 288 detik
10. Young - Vacations - 271 detik

Data Setelah Quick Sort (Durasi Asc):
1. Poor Grammer - Roar - 225 detik
2. Bimbang - Melly Goeslaw - 226 detik
3. Sing To You - Silodinasti - 233 detik
4. Mari Bercerita - Payung Teduh - 241 detik
5. Kursi Goyang - Fourtwny - 257 detik
6. Young - Vacations - 271 detik
7. Ode To the Mets - The Strokes - 288 detik
8. Be For You Go - Lewis Capaldi - 334 detik
9. Hysteria - Muse - 344 detik
10. Aku Pasti Kembali - Pasto - 440 detik
```

